

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2
1ra práctica (tipo b)
Segundo Semestre 2026

Indicaciones Generales:

- Duración: 110 minutos.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear **variables globales**, **NI OBJETOS** (con excepción de los elementos de `iostream`, `omanip` y `fstream`). **NO PUEDE UTILIZAR LA CLASE `string`**. Tampoco se podrán emplear las funciones `malloc`, `realloc`, `memset`, `strtok` o `strdup`, igualmente no se puede emplear cualquier función contenida en las bibliotecas `stdio.h`, `cstdio` o similares y que puedan estar también definidas en otras bibliotecas. **NO PODRÁ EMPLEAR PLANTILLAS EN ESTE LABORATORIO**
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN Estricto DISEÑO DESCENDENTE. Cada función NO debe sobrepasar las 20 líneas de código aproximadamente. El archivo `main.cpp` solo podrá contener la función `main` de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo `main.cpp` deberá colocar un comentario en el que coloque claramente su nombre y código, de no hacerlo se le descontará 0.5 puntos en la nota final.
- El código comentado NO SE CALIFICARÁ. De igual manera NO SE CALIFICARÁ el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestren resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.

SE LES RECUERDA QUE, DE ACUERDO CON EL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

- Puntaje total: 20 puntos.

INDICACIONES INICIALES

Cree un proyecto de C++ en CLION siguiendo estrictamente las indicaciones que a continuación se detallan:

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"Lab01_2026_2_CO_PA_PN"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará los proyectos solicitados en la prueba.**
- En cada proyecto que desarrolle deberá crear una carpeta denominada **"Bibliotecas"** y allí coloque la biblioteca de funciones que necesite. Dentro de la carpeta **"cmake-build-debug"** cree las carpetas **"ArchivosDeDatos"** y **"ArchivosDeReporte"** y allí respectivamente coloque el archivo de datos que se le proporcionará y el archivo del reporte solicitado en la prueba. De no colocar esto se descontará un punto en cada proyecto.

Cuestionario:

La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 1 del curso: "Funciones y alcance de variables". En este laboratorio se desarrollará una **biblioteca estática de funciones** en la que se implementen sobrecargas de operadores y funciones que le permitan solucionar el

problema planteado.

En la carpeta solicitada anteriormente, cree **dos carpetas** denominadas "**CrearBiblioteca_Parte1**" y "**UsarBiblioteca_Parte2**", allí colocará los proyectos solicitados en las partes 1 y 2 respectivamente de este laboratorio. **DE NO COLOCAR ALGUNO DE ESTOS REQUERIMIENTO SE LE DESCONTARÁ 3 PUNTOS DE LA NOTA FINAL. NO SE HARÁN EXCEPCIONES.**

PARTE 01 (10 puntos): CREACIÓN DE LA BIBLIOTECA ESTÁTICA

Una empresa desea conocer el stock disponible de sus productos, tanto en **unidades individuales** como en **empaques destinados a la venta al por mayor**. Para ello, necesita generar un reporte por cada almacén, que indique la cantidad de unidades disponibles de cada producto y, cuando corresponda, la cantidad de **empaques completos** que pueden formarse a partir de dichas unidades. Una vez realizada la conversión de las unidades disponibles en empaques, puede ocurrir que algunas unidades no sean suficientes para formar un empaque completo. Estas unidades restantes se consideran **picking** y también deben ser mostradas en el reporte. Por ejemplo, si un producto tiene un stock de **27 unidades** y cada empaque contiene **6 unidades**, se pueden formar **4 empaques completos** y quedan **3 unidades de picking**.

A continuación, definiremos las estructuras necesarias para manejar la información descrita anteriormente:

struct Producto servirá para almacenar la información de un producto.

struct Empaque almacena la información necesaria para que un producto sea empacado, como la cantidad de unidades que forman un empaque y el nombre del empaque.

struct ProductoStock contiene la información de cada producto como su código, nombre del producto, stock en unidades y adicionalmente su conversión a empaques completos y unidades picking si fuera el caso.

struct Almacen contiene la información del almacén como su código, cuantos productos almacena y un arreglo de **ProductoStock** para registrar los productos y su conversión respectiva.

Se solicita que desarrolle una biblioteca estática en la cual se defina una serie de operadores sobrecargados que permitirá manejar las estructuras de datos creadas.

Las operaciones que la biblioteca estática permitirá realizar a través de sobrecargas de operadores se definen a continuación:

Lectura:

- Sobrecargando el operador **>>** de modo que permita leer los datos de **un** producto de un archivo de tipo CSV. La operación **arch >> producto;** involucrará una variable de archivo y una variable de tipo "**struct Producto**". Una línea de archivo tendrá la siguiente forma:

1, Avena Pacifico Tipo 17, UNIDAD
(código del producto, nombre del producto, unidad)

- Sobrecargando el operador **>>** de modo que permita leer los datos de **un** empaque correspondiente a un producto de un archivo de tipo CSV. La operación **arch >> empaque;** involucrará una variable de archivo y una variable de tipo "**struct Empaque**". Una línea de archivo tendrá la siguiente forma:

1, PAQ25,25
(código del producto, nombre del empaque, unidades por empaque)

➤ Operaciones:

- Sobrecargando el operador **!=** de modo que permita buscar **un** almacén en **un** arreglo de almacenes. La operación **ArrAlmacen!=Almacen;** buscará el almacén (**struct Almacen**) en el arreglo de almacenes, esta función retorna el índice del almacén buscado dentro del arreglo, si no lo ubica puede retornar -1. Como no se puede trabajar con la cantidad de datos en el arreglo de almacenes, deberá tener como último dato un registro con el código con valor nullptr.
- Sobrecargando el operador **+=** de modo que permita agregar **un** producto_stock a **un** almacén. La operación **Almacen+=ProductoStock;** agregará el producto_stock (**struct ProductoStock**) al

almacén correspondiente (**struct Almacen**). Cada vez que se añade un producto_stock al almacén, debe colocarse en el arreglo que le corresponde. El número de productos que tiene el almacén debe incrementarse.

- Sobrecargando el operador <= de modo que permita actualizar los empaques del arreglo producto_stock de un almacén. La operación **Almacen <= ArrEmpaque**; **ArrEmpaque** es un arreglo de **struct Empaque**. Este proceso deberá recorrer el arreglo de producto_stock del almacén y para cada producto actualizar la cantidad de productos empacados y la cantidad de picking de acuerdo con la conversión que le corresponde. Estos campos inicialmente estaban en 0.
- Sobrecargando el operador <= de modo que permita actualizar los datos del producto del arreglo producto_stock de un almacén. La operación **Almacen <= ArrProducto**; **ArrProducto** es un arreglo de **struct Producto**. Este proceso deberá recorrer el arreglo de producto_stock del almacén y para cada elemento actualizar en nombre del producto. Este campo inicialmente estaba en nullptr.
- Sobrecargando el operador && de modo que permita actualizar el empaque de un producto_stock. La operación **Producto_Stock && ArrEmpaque**; **ArrEmpaque** es un arreglo de **struct Empaque**. Este proceso deberá recorrer el arreglo de empaques y para el **producto_stock** asignado actualizar la cantidad de productos empacados y la cantidad de picking de acuerdo con la conversión que le corresponde. Estos campos inicialmente estaban en 0.
- Sobrecargando el operador || de modo que permita actualizar los datos de un producto_stock. La operación **Producto_Stock || ArrProducto**; **ArrProducto** es un arreglo de **struct Producto**. Este proceso deberá recorrer el arreglo de productos y para el **producto_stock** asignado actualizar el nombre del producto. Este campo inicialmente estaba en nullptr.

➤ Impresión:

- Sobrecargando el operador << de modo que permita imprimir la información de un producto_stock. La operación **arch << producto_stock**; permitirá imprimir en un archivo de textos los datos contenidos en una variable de tipo "**struct ProductoStock**". El formato será el siguiente:

```
Codigo del Producto: 433
Nombre del Producto: Frijol Misti Tipo 18
Stock Total: 1206

Stock en Paquetes:
Paquetes: 12
Unidades: 6
```

- Sobrecargando el operador << de modo que permita imprimir la información de un almacén. La operación **arch << almacen**; permitirá imprimir en un archivo de textos los datos contenidos en una variable de tipo "**struct Almacen**". El formato será el siguiente:

```
Almacen: ALM02
Numero de Productos:139
=====
Detalle de Productos:
=====

Codigo del Producto: 1
Nombre del Producto: Avena Pacifico Tipo 17
Stock Total: 1409

Stock en Paquetes:
Paquetes: 56
Unidades: 9
=====

Codigo del Producto: 2
...
```

Los datos deben aparecer con un formato adecuado y alineados correctamente).

Consideraciones:

La solución debe contemplar la elaboración de:

- 1) un proyecto de implementación y prueba de las sobrecargas, denominado **"Fuentes_Biblioteca_2026_2"**, en este mismo proyecto puede generar la biblioteca estática (.a). También puede generar un proyecto tipo biblioteca estática para esta tarea. Debe presentarse como parte de la solución.
- 2) Si desea puede crear un proyecto donde se pruebe la biblioteca ya compilada **"Prueba_Biblioteca_2026_2"**. La prueba de las sobrecargas puede ser hechas lo más simple posible pero que muestre claramente que son correctas.
- 3) Todos los proyectos deberán colocarse en la carpeta **"CrearBiblioteca_Parte1"**.

EN LAS PRUEBAS PARA LOS PROYECTOS SE PUEDEN HACER DIRECTAMENTE EN LA FUNCIÓN MAIN, SE PUEDE HACER USO DE "HARD CODE" Y SE PUEDE SOBREPASAR LA CANTIDAD EL NUMERO DE LÍNEAS, SOLO EN ESTA PARTE DE LA PRUEBA.

EN LA BIBLIOTECA ESTÁTICA, NO PUEDE DESARROLLAR OTRAS FUNCIONES O SOBRECARGAS ADICIONALES A MENOS QUE ESTÉN DIRECTAMENTE RELACIONADAS A LAS SOBRECARGAS SOLICITADAS, COMO POR EJEMPLO LEECADENA.

San Miguel, 04 de septiembre del 2026.